

WHIEL: A Neurosymbolic Platform for Certified Relational Program Verification

Jesse Comer*
University of Pennsylvania
Philadelphia, Pennsylvania, USA
jacomer@engineering.upenn.edu

Fangzhu Shen*
Duke University
Durham, North Carolina, USA
fangzhu.shen@duke.edu

Shuo Zhang*
Duke University
Durham, North Carolina, USA
shuo.zhang2@duke.edu

Mayur Naik
University of Pennsylvania
Philadelphia, Pennsylvania, USA
mhnaik@engineering.upenn.edu

Sudeepa Roy
Duke University
Durham, North Carolina, USA
sudeepa@cs.duke.edu

Val Tannen
University of Pennsylvania
Philadelphia, Pennsylvania, USA
val@engineering.upenn.edu

ABSTRACT

We present the WHIEL (a contraction of Whi(le-R)el) system for LEAN-certified reasoning about imperative relational programs. The system can encode containment and equivalence problems for DATALOG, its extensions with negation, and the core fragment of graph query languages. It can also represent optimized or incremental evaluation algorithms that are not expressible in the original query language, as well as first-order specifications of integrity constraints, schema mappings, and views arising in data integration, exchange, and sharing. WHIEL’s program states are database instances, and its imperative structure enables compositional Hoare-style reasoning with first-order relational specifications, using prophecy variables and leveled invariants to support inductive arguments. WHIEL supports neurosymbolic discovery of loop invariants whose verification conditions can be checked using the VAMPIRE first-order theorem prover. Counterexamples are fed back into the neurosymbolic pipeline, while proposed solutions are translated into Lean proof certificates. To validate the WHIEL system, we evaluate its performance on a curated benchmark suite. In summary, we envision the WHIEL system as a tool for tackling mathematically demanding, typically undecidable problems involving complex query and constraint languages, with a high degree of trust in its output.

PVLDB Reference Format:

Jesse Comer, Fangzhu Shen, Shuo Zhang, Mayur Naik, Sudeepa Roy, and Val Tannen. WHIEL: A Neurosymbolic Platform for Certified Relational Program Verification. PVLDB, 20(1): XXX-XXX, 2027. doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://github.com/JesseComer/whiel>.

*The first three student authors are listed alphabetically by last name with contributions detailed in the full version [59] for their dissertation credit.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097. doi:XX.XX/XXX.XX

1 INTRODUCTION

The rapidly growing capabilities of LLMs introduce numerous opportunities for applications in the management of data. A database developer might use an LLM to generate an optimized evaluation strategy for a recursive query, an algorithm for incrementally updating views, or a query rewritten using views. A correct solution to such requests can improve query evaluation, reduce computation as a database is updated, or enable applications to work across schemas in data integration, exchange, and sharing. Fernandez et al. [23] predict that LLMs will revolutionize how we approach such hard, often undecidable, database problems.

But how can a developer be sure that a solution proposed by an LLM to such problems is correct? An incorrect query rewrite or faulty view maintenance algorithm can lead to wrong or missing answers, which can have serious consequences for users, who depend on the accuracy of query results. One approach is to manually review the LLM-proposed solution; another is to test query or view maintenance algorithm on sample databases. However, while such approaches might expose mistakes, they do not establish correctness of a rewritten query across all databases. Furthermore, an argument showing that even a short recursive program computes the right results can require a substantial amount of mathematical sophistication. Evidence [17] suggests that coding agents leave a bottleneck at the point of human review; that is, the burden has shifted from *writing code* to *confirming that the code is correct*. In the case of checking LLM-generated solutions to hard database problems, the time and expertise required for such a review can be impractical. This leaves developers with the choice to either expend the requisite effort, or to trust a solution which has not been independently confirmed.

However, recent successes in AI-assisted mathematics [53] point toward another approach: asking the LLM to *prove* correctness of its solution in an interactive theorem prover such as LEAN [16]. LEAN provides a small, trusted *kernel*; given a carefully formalized theorem statement and a *proof term*, the kernel checks whether or not the proof is valid. Given such a formalization, a human reviewer needs only to inspect the mathematical definitions and theorem statements. However, producing these definitions on each invocation of an LLM to solve a hard database problem means checking the formalization each time, leaving a still substantial burden on the reviewer. Furthermore, such requests can require highly capable models, resulting in significant financial cost in order

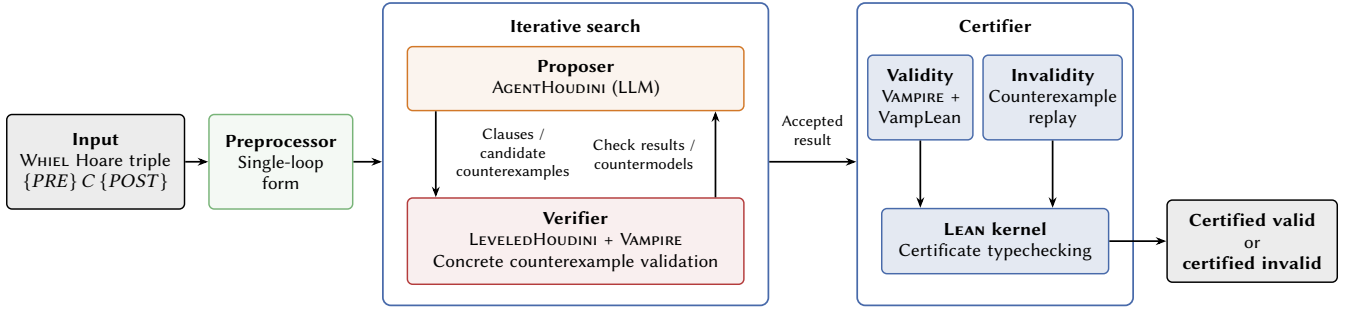


Figure 1: Overview of the WHIEL system. Search seeks an invariant satisfying the ladder rule or a concrete counterexample. The certifier produces a LEAN-checked proof of validity or invalidity for the original input triple.

to produce a solution. One idea to reduce this time and financial cost is to produce reusable definitions one time; however, it still leaves success or failure up to the LLMs ability to construct proofs across a wide range of problem-specific formalizations.

We believe that a simpler approach is to target a single general and hard problem into which all of the others – query equivalence, containment, rewriting, incremental updating, and more – can be encoded. As this single general problem we choose *verification of imperative relational programs*. Program verification is a well-studied area with a developed mathematical theory and powerful tools, such as SAT/SMT solvers, automated and interactive theorem provers, capable of coping practically with the intractable and undecidable nature of the problem. Using such tools, instead of a variety of insights for proving the correctness of proposed solutions to a variety of problems, we only need insights for “guessing” loop invariants. We use LLMs for this and then our platform can “check” these guesses and certify the correctness proofs.

Our Contributions

We introduce the WHIEL system, a neurosymbolic platform for certified verification of *relational programs*. In WHIEL’s relational programming language, *program states* are database instances, updated by assigning the results of relational algebra (RA) expressions to relation-valued variables. The system is designed to solve the *WHIEL Hoare validity problem* which, informally, asks whether or not a relational program satisfies a given first-order specification of its correctness (overall formalized as a *Hoare triple*; see Equation 1). The WHIEL Hoare validity problem serves as the common target proposed above: it is highly general, capable (see Section 3) of simulating not only recursive queries written in DATALOG [12], in its extensions with negation [1], or in the *cores* of graph query languages [26], but also their evaluation strategies. It can also represent DATALOG containment and equivalence with respect to constraints, optimized or incremental algorithms, correctness of view-based query rewrites, schema mappings, and problems arising in data integration, exchange, and sharing. Through encodings, WHIEL aligns with the vision of Castro Fernandez et al. [23], who predicted that LLMs might overcome the “ceiling” for existing automation of these hard problems. WHIEL also addresses one of the key challenges of this vision: ensuring that the LLM’s proposed solutions are correct.

The primary challenge in solving the WHIEL Hoare validity problem is the synthesis of *loop invariants*: formal assertions used to prove correctness properties of program loops. By pre-processing arbitrary program written in WHIEL’s relational language to *single-loop form*, WHIEL reduces this issue to the problem of synthesizing a single invariant for the pre-processed program. It then implements an LLM-powered synthesis search. Failing that, WHIEL can produce counterexamples to the validity of Hoare triples, specifically, initial program states from which execution terminates in a state violating the specification. Such counterexamples prove that the program does *not* satisfy the specification.

Consider a developer seeking a view-based rewriting of a DATALOG query. We envision a workflow (Figure 2) in which a coding agent, LLM_{Solve}, proposes a Datalog program over the view schema. Then, the equivalence of this rewriting to the original query is encoded as an instance of the WHIEL Hoare validity problem, which is passed into the WHIEL system. The WHIEL system utilizes a powerful *guess-and-check* methodology (Section 5), with another agent, LLM_{WHIEL}¹, generating candidate loop invariants or candidate counterexamples that are then checked by WHIEL’s symbolic apparatus. WHIEL outputs a proof if it confirms that LLM_{Solve}’s rewriting is correct; if it returns a counterexample, then LLM_{Solve} can use it to revise its attempted rewrite.

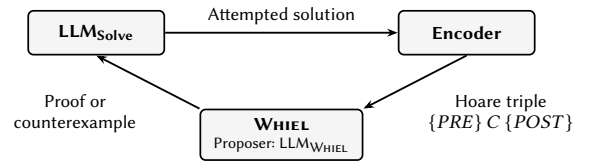


Figure 2: Envisioned workflow for solving a hard database problem with feedback from WHIEL.

WHIEL (Figure 1) combines a flexible *proposer*, an automated symbolic *verifier*, and an independent, LLM-free *certifier*. During iterative search, the verifier checks proposer-provided candidate invariants or counterexamples using our LEVELEDHOUDINI algorithm, a variant of the classical HOUDINI algorithm [24]. This algorithm accepts sets of first-order expressible assertions which may contain prophecy variables (Definition 4.4) representing the terminal

¹We refer to this agent as AGENTHOUDINI in Figure 1 and Section 5.

values of existing program variables. These assertions are automatically organized by `LEVELEDHOUDINI` into *leveled assertions* using a pruning process involving `VAMPIRE` [38] calls and `LEAN`-based operations. The verifier provides feedback for further proposals until a proposal is accepted or the resource budget is exhausted. Properties of terminal values established at lower levels support proofs of further properties at higher levels, while the resulting *verification conditions* remain first-order.

The final component of the `WHIEL` system is the *certifier*, which creates, without LLM-assistance, `LEAN` certificates that a given Hoare triple is valid or invalid. Invariants are certified using our `Lean` library combined with `Vampire` proofs reconstructed via `VAMPLEAN` [9], while counterexamples are certified by a relational program interpreter and model-checker implemented in `LEAN`. Depending on the application, certification is an optional step which reduces the *trusted computing base* of the answer to only the `LEAN` kernel, removing the need to trust the LLM proposer, verifier implementation, or even `VAMPIRE`.

Our experiments (Section 6) demonstrate the value and power of `WHIEL`’s neurosymbolic architecture. With GPT-5.5, `WHIEL`’s iterative search process is able to solve 73 of 86 of the tasks in our curated benchmark. In contrast, our two separate baseline methods, one which attempts to directly generate a `LEAN` proof without help from our symbolic harness (`DIRECTLEAN`), and another which makes only one proposal but does not receive feedback or iterate (`WHIEL-ONEROUND`), solve 20 and 46 benchmark examples, respectively. In addition to the success rate, `WHIEL` solves more tasks at an average token cost of \$0.49, compared with \$2.08 for `DIRECTLEAN`. The first comparison demonstrates the value of the symbolic harness; the second demonstrates the further benefit of iterative search. `WHIEL`’s 600-second budget covers search and runtime verification; certification is measured separately, since it occurs after an answer has already been accepted by the verifier.

Beyond data management, `WHIEL`’s certified verification can enable reliable LLM-assisted development of relational programs, whose application areas continue to grow: these include declarative networking and network analysis [42, 64, 65], static program analysis [34, 63], and enterprise and knowledge-graph applications through systems such as `Rel` [3] and `Vadalog` [4]. `Scallop` and `Lobster` apply the paradigm to neurosymbolic programming [8, 41], alongside work on declarative machine learning [60]. Differential dataflow [48] and incremental and dynamic graph processing [7, 21, 22] address efficient evaluation over changing data.

The novel contributions of this paper are:

- (1) **A common verification target.** We introduce the `WHIEL` Hoare validity problem (Section 2), and explain how to encode database problems, including recursive query equivalence and containment, correctness of evaluation strategies, incremental view maintenance, and view-based query rewriting (Section 3).
- (2) **Automated structuring of ladder invariants** (Section 4). We introduce *ladder invariants* and their compact *leveled assertion* representations, supporting nested inductive arguments in which properties of a loop’s final state help establish further properties. Our `LEVELEDHOUDINI` algorithm automatically organizes proposed clauses into levels reflecting this inductive structure while keeping verification conditions first-order.

- (3) **An implemented neurosymbolic verification system** (Section 5). We develop `WHIEL`, combining an agent proposer with a symbolic verifier that checks candidate invariants and counterexamples and provides feedback for iterative search. An independent certifier constructs `LEAN` proofs of validity or invalidity without further LLM assistance.
- (4) **A certified benchmark and experimental evaluation** (Section 6). We provide a benchmark of 86 relational verification tasks with certified reference answers. Our evaluation measures the benefits of `WHIEL`’s iterative search relative to direct `LEAN` proof generation and single-proposal baselines, and separately reports certification success, time, and memory usage.

Related work

Theoretical work on database program verification goes back as far as the work by Casanova and Bernstein [11] in 1980 who proposed a formal system for reasoning about programs accessing a relational databases through data definition and manipulation languages. Berghammer et al. [6] verify relational programs over binary relations using manually-supplied invariants, discharging their verification conditions using the automated first-order theorem prover `Prover9` [46]. Itzhaky et al. [33] verify loop-free scripts over a fragment of SQL using weakest precondition computations, with their resulting verification conditions falling within a decidable fragment of first-order logic (FOL). Mediator [62] verifies equivalence between pairs of programs implementing loop-free update transactions and nonrecursive queries, with invariants generated using clause templates and `HOUDINI`-style pruning. Beyond database and relational verification, `LOOPY` [36] and the work of Pirzada et al. [54] utilize LLMs as candidate invariant generators, with the former using `HOUDINI`-style pruning and the latter using `VAMPIRE` to avoid unrolling loops during bounded model-checking.

`VERIEQL` [32] proves *bounded* equivalence of SQL queries. Symbolic tools for proving unbounded equivalence of SQL queries include `SQLSOLVER` [19], `QED` [61], and `COSETTE` [13], with the latter even providing proofs verified in the proof assistant `Coq` (now `Rocq` [56]). `GRAPHQE` [55] proves equivalence for `CYPHER` [25] queries, while `GRAPHITI` [31] proves equivalence between `CYPHER` and SQL queries. Even more recent work studies verified LLM-guided SQL rewrites [40], while a recent preprint introduces `Logos` [37], a tool for verifying SQL rewrites in `Rocq`.

Prophecy variables support reasoning about future program behavior [35, 39]. `WHIEL`’s prophecy termination condition is reminiscent of the use of prophecy variables to establish a property whenever predicted values agree with actual program values [44, 47]. We introduce the `LEVELEDHOUDINI` algorithm to automatically organize candidate clauses into stages of a nested inductive argument. Within each stage, our algorithm applies the annotation-pruning framework of Flanagan et al. [24] to identify mutually-supporting clauses under assumptions from earlier stages.

2 THE WHIEL HOARE VALIDITY PROBLEM

2.1 Preliminaries

Database terminology. We write Γ to denote database schemas, which are finite collections of relation symbols, each with an assigned natural number *arity*. A relation symbol is *nullary* if it has

arity 0. A database instance over Γ is a map from the symbols of Γ to finite relations of appropriate arity over a fixed data domain.

Datalog is a declarative query language. *DATALOG programs* are written as sets of rules. Rules are evaluated by matching the right-hand side of the rule to a database instance, and then deriving the tuple on the left-hand side. Evaluation repeatedly applies these rules until no new tuples can be derived. As an example, the following program computes the transitive closure T of a binary relation E :

$$\begin{aligned} T(x, y) &\leftarrow E(x, y) \\ T(x, y) &\leftarrow E(x, z), T(z, y). \end{aligned} \quad (\mathbf{TC}_{\text{DATALOG}})$$

Computability. A decision problem is *recursively enumerable* (RE) if there is an algorithm which enumerates its positive instances. The class **coRE** consists of problems whose complements are in RE. A problem is complete for a class if it belongs to that class and if any problem in the class can be reduced to it via *many-one reductions*; that is, computable maps that preserve the answers. Many hard decision problems in databases, like those mentioned in the introduction, are **coRE**-complete. Consequently, no sound, effective procedure can enumerate all positive instances. An *oracle* for a decision problem is a function that returns the correct answer for all inputs to the problem. An oracle is a construct for theoretical analysis; the function need not be computable.

2.2 WHIEL Programs and Hoare Triples

WHIEL *commands* transform database instances via assignments to relation-valued variables over a fixed *execution schema*. *States* for such commands are database instances over the execution schema; that is, states are maps from the schema symbols to finite relations of appropriate arity. Assignments update variables using RA expressions; guards of **while** loops and **if-then-else** conditionals are WHIEL *assertions*: Boolean combinations of inclusions and equalities between RA expressions. The formal syntax is given by

$$\begin{aligned} \varphi &::= i = j \mid i = c \mid \neg\varphi \mid \varphi \wedge \varphi \mid \varphi \vee \varphi \\ E &::= \top \mid \emptyset \mid R \mid \{d\} \mid \sigma_\varphi E \mid \pi_L E \mid E \times E \mid E \cup E \mid E - E \\ A &::= \text{true} \mid \text{false} \mid E \subseteq E \mid E = E \mid \neg A \mid A \wedge A \mid A \vee A \\ C &::= X := E \mid C; C \mid \text{skip} \mid \text{if } A \text{ then } C \text{ else } C \mid \text{while } A \text{ do } C \text{ end} \end{aligned}$$

Here, i and j denote tuple indices, φ denotes selection conditions, R and X are relation symbols, d denotes a domain constant, E denotes RA expressions, A denotes WHIEL assertions, and C denotes WHIEL commands. RA expressions are evaluated in states (i.e., database instances) as usual, while *assertions* are either satisfied or falsified. We write $E(s)$ to denote the result of evaluating RA expression E in state s . For assertions, $E_1 \subseteq E_2$ is satisfied in s (notation: $s \models E_1 \subseteq E_2$) if $E_1(s) \subseteq E_2(s)$, and similarly for equalities. The semantics of assertions are extended to Boolean combinations in the natural way. Given a state s , we write $s \models A$ if A evaluates to true in s (under the natural semantics for the guard syntax), in which case we say that A is *satisfied* in s .

Assertion A *entails* assertion B (notation: $A \models B$) if every state satisfying A also satisfies B ; in this case we say that the *entailment* $A \models B$ is *valid*. Using *active domain semantics* [1], Codd's theorem [14] implies that WHIEL assertions can be translated to sentences of first-order logic (FOL), and that every sentence of

FOL is expressed by some WHIEL assertion. WHIEL includes a Lean-formalized translation from assertions to first-order logic, enabling the system to check entailments between assertions using VAMPIRE.

WHIEL is endowed in LEAN with a formalized big-step semantics, mapping input states to output states. Assignments update assigned variables to the result of evaluating RA expressions; $C_1; C_2$ executes C_1 followed by C_2 ; **skip** does nothing; and **while** and **if-then-else** have their usual guarded semantics. The semantics are partial (i.e., the output state is sometimes undefined) due to the potential for nonterminating **while** loops.

The DATALOG program for transitive closure (Program $\mathbf{TC}_{\text{DATALOG}}$) can be simulated in WHIEL (see Program $\mathbf{TC}_{\text{WHIEL}}$). Borrowing terminology from DATALOG, we refer to relational variables which appear on the left-hand side of some assignment as *intensional database predicates* (IDBs), and all other relational variables as *extensional database predicates*. Given a WHIEL command C , we write $\text{IDB}(C)$ (resp. $\text{EDB}(C)$) to denote its intensional (resp. extensional) database predicates. For example, in the program below, we have that $\text{IDB}(\mathbf{TC}_{\text{WHIEL}}) = \{S, T\}$, while $\text{EDB}(\mathbf{TC}_{\text{WHIEL}}) = \{E\}$.

$$\begin{aligned} &\text{while } T \neq S \text{ do} \\ &\quad T := S \\ &\quad S := E \cup E \circ T \\ &\text{end} \end{aligned} \quad (\mathbf{TC}_{\text{WHIEL}})$$

We write $E \circ T$ as shorthand for relational composition, which is definable in RA using a select-project-join query. Note that $\mathbf{TC}_{\text{WHIEL}}$ uses an *auxiliary variable*² S to capture the next value for T . Under appropriate assumptions on the initialization of S and T (see Equation 2), the program computes the transitive closure of E . Auxiliary variables are not strictly necessary for simulating DATALOG programs in WHIEL, but we include S for expository purposes.

Inputs to the WHIEL system are *Hoare triples*, written

$$\{PRE\} C \{POST\}, \quad (1)$$

where PRE and $POST$ are *assertions* and C is a WHIEL command. A Hoare triple is *valid* if, for all terminating runs of C starting from a state satisfying PRE , the halting state satisfies $POST$. The triple is *invalid* otherwise. Thus, providing a single state which satisfies PRE and for which C terminates in a state falsifying $POST$ suffices to prove the triple invalid. The goal of the WHIEL system is to determine whether or not a given Hoare triple is valid; this is the *WHIEL Hoare validity problem*.

THEOREM 2.1. *WHIEL Hoare validity is coRE-complete.*

As an example, consider the following Hoare triple:

$$\{T = \emptyset \wedge S = E\} \quad \mathbf{TC}_{\text{WHIEL}} \quad \{T \circ T \subseteq T\}. \quad (2)$$

This asserts that, assuming that the IDB T is initially empty while S is initialized to E , the final value of T after running the $\mathbf{TC}_{\text{WHIEL}}$ program is transitive.

3 APPLICATIONS OF THE WHIEL SYSTEM

WHIEL can be applied to hard problems in databases by encoding them as instances of the WHIEL Hoare validity problem. This may initially seem unsurprising, since the problem's **coRE**-completeness

²A variable which is not part of the “intended” output, but supports the computation.

Query T	Views	Rewriting R
$T(x, y) \leftarrow E(x, \text{red}, y)$	$\text{RedPV}(x, y) \leftarrow E(x, \text{red}, y)$	$R(x, y) \leftarrow \text{RedPV}(x, y)$
$T(x, y) \leftarrow E(x, \text{blue}, y)$	$\text{RedPV}(x, z) \leftarrow E(x, \text{red}, y), \text{RedPV}(y, z)$	$R(x, y) \leftarrow \text{BluPV}(x, y)$
$T(x, z) \leftarrow E(x, \text{red}, y), T(y, z)$	$\text{BluPV}(x, y) \leftarrow E(x, \text{blue}, y)$	$R(x, z) \leftarrow R(x, y), R(y, z)$
$T(x, z) \leftarrow E(x, \text{blue}, y), T(y, z)$	$\text{BluPV}(x, z) \leftarrow E(x, \text{blue}, y), \text{BluPV}(y, z)$	

entails that *any* problem in the class of **coRE**, including many database problems, can be reduced to **WHIEL** Hoare validity. However, it is striking how *natural* and *compact* these encodings are, and how efficiently they can be constructed.

Encoding DATALOG. Consider the DATALOG *path system* query [15]:

$$\begin{aligned} T(x) &\leftarrow A(x), \\ T(x) &\leftarrow R(x, y, z), T(y), T(z). \end{aligned} \quad (\mathbf{PS}_{\text{DATALOG}})$$

Here, A denotes a set of initially accessible nodes, while a tuple $R(x, y, z)$ says that node x is accessible if both y and z are accessible. We will use this example to illustrate **WHIEL**'s ability to express different evaluation strategies for DATALOG. Define the shorthand

$$H(X, Y) \equiv \pi_{[0]} (\sigma_{1=3 \wedge 2=4} ((R \times X) \times Y))$$

The following two programs simulate *naive* and *semi-naive* evaluation [1, Section 13.1], respectively:

$\mathbf{PS}_{\text{NAIVE}}$	$\mathbf{PS}_{\text{SEMINAIVE}}$
$T := A$	$U := A; \Delta := A$
$S := A \cup H(T, T)$	while $\Delta \neq \emptyset$ do
while $T \neq S$ do	$N := (H(\Delta, U) \cup H(U, \Delta)) \setminus U$
$T := S$	$U := U \cup N$
$S := A \cup H(T, T)$	$\Delta := N$
end	end

Encoding Graph Queries. Problems stated over the *cores* of property-graph query languages such as GQL and SQL/PGQ are also naturally encoded into the **WHIEL** Hoare validity problem. Indeed, the core of such languages comprises graph-pattern matching (expressible with (even linear) DATALOG) followed by relational operations [10, 26]. Han and Ives exploit this perspective to implement query rewriting with views using relational techniques [29].

We suggest that the example in Equation 3 illustrates this connection convincingly, without the need to use graph language syntax. The query T computes reachability along all edges, red or blue; RedPV and BluPV compute the red-only and blue-only path views. The rewriting R recovers T by composing paths supplied by these views. Although this example is simple, deciding whether an equivalent DATALOG rewriting using given views exists is undecidable in general [20]. We have encoded this example in **WHIEL** using separate binary relations for red and blue edges, and verified and certified the equivalence of the original query and its rewriting.

More generally, **WHIEL** can express interesting graph computations such as computation of the *attractor set* for two-player *reachability games* on finite graphs [45]. In such games, we are given a graph (V, E) with no dead ends and subsets of vertices $P_a, \text{Target} \subseteq V$. In such a game, there is an *attacker* whose goal is to reach the *Target* vertices. The goal of the *defender* is to prevent the attacker from reaching the *Target* set. The attacker may choose

the next node from vertices in P_a ; other nodes belong to the defender. The attractor of the game is the set of nodes W from which the attacker has a strategy to force the game position to reach the *Target* set, irrespective of the moves chosen by the defender. Define $\text{Pred}(X) := \pi_{[0]} (\sigma_{1=2} (E \times X))$. The following monotone operator on the powerset of vertices has, as its least fixed point, the *attractor set* of the graph to which it is applied:

$$F(X) = \text{Target} \cup (P_a \cap \text{Pred}(X)) \cup ((V \setminus P_a) \setminus \text{Pred}(V \setminus X)).$$

Using this operator, the corresponding **WHIEL** command is

```

 $W := \emptyset; Z := F(W)$ 
while  $W \neq Z$  do
   $W := Z$ 
   $Z := F(W)$ 
end

```

($\mathbf{RG}_{\text{WHIEL}}$)

Several interesting correctness properties of this command can be expressed in **WHIEL**'s assertion language. Let Σ denote some collection of first-order constraints on the input graph, and let I be a subset of vertices. To check whether these constraints guarantee that I is *safe*, meaning that the attacker cannot win from any vertex in I , we can ask whether the following Hoare triple is valid:

$$\{\Sigma\} \quad \mathbf{RG}_{\text{WHIEL}} \quad \{I \cap W = \emptyset\}.$$

Encoding Other Database Problems. We have already seen that **WHIEL** can encode DATALOG queries and express equivalence between them; even extensions with negation can be expressed using the difference operator. More generally, we can encode first-order constraints (such as tuple- or equality-generating dependencies) in the precondition, and compare the corresponding output relations by equality or inclusion. As with DATALOG evaluation strategies, **WHIEL** can also express correctness of incremental view computation algorithms by first computing the original view and auxiliary state, then applying the input update and maintenance algorithm, and comparing the maintained result with full recomputation on the updated database. Schema mappings can also be expressed as first-order constraints. For view-based rewritings, we first compute the view, and then compare the result of the original query with the result of the rewritten query applied to the view schema.

4 WHIEL'S VERIFIER

In this section, we explain the mathematical theory behind **WHIEL**'s verifier. For this, we return to our expository example, **TC_{WHIEL}**.

4.1 Hoare Logic and Verification Conditions

The key challenge in solving the **WHIEL** Hoare validity problem is the identification of *loop invariants*; that is, given a Hoare triple

$$\{PRE\} \text{ while } G \text{ do } B \text{ end } \{POST\}, \quad (4)$$

to identify an assertion I satisfying the *verification conditions*

$$\begin{array}{ll} PRE \models I, & (\mathbf{init})_I \\ \{I \wedge G\} B \{I\} \text{ is valid, and} & (\mathbf{maint})_I \\ I \wedge \neg G \models POST. & (\mathbf{term})_I \end{array}$$

We refer to these as the *initialization*, *maintenance*, and *termination* conditions, respectively. Note that $(\mathbf{init})_I$, $(\mathbf{maint})_I$, and $(\mathbf{term})_I$ depend on PRE , $POST$, G , and B from the Hoare triple in Equation 4. We leave this dependence on the Hoare triple implicit. We call I *inductive* if $(\mathbf{init})_I$ and $(\mathbf{maint})_I$ hold. Intuitively, these three conditions assert that the assertion I is satisfied at the start of the loop $((\mathbf{init})_I)$, after each loop iteration $((\mathbf{maint})_I)$, and that I together with the negation of the loop guard implies $POST$ $((\mathbf{term})_I)$.

PROPOSITION 4.1. *If there exists an assertion I satisfying $(\mathbf{init})_I$, $(\mathbf{maint})_I$, and $(\mathbf{term})_I$, then the Hoare triple in Equation 4 is valid.*

Since PRE , $POST$, I , and G can be translated to FOL, the $(\mathbf{init})_I$ and $(\mathbf{term})_I$ checks are effectively FOL entailments. To express $(\mathbf{maint})_I$ as an FOL entailment, we use the *weakest liberal precondition of I with respect to B* , which is the set of states from which all terminating runs of B halt in a state satisfying I [18]. This set is *expressible* if there exists a WHIEL assertion $\mathbf{wlp}(I, B)$ which is satisfied in all and only these states.

PROPOSITION 4.2. *For an assertion I and loop-free³ command B , $\mathbf{wlp}(I, B)$ exists and is computable from I and B ; and for every WHIEL assertion G , $\{I \wedge G\} B \{I\}$ is valid if and only if $I \wedge G \models \mathbf{wlp}(I, B)$.*

For an assignment $X := E$, we have that $\mathbf{wlp}(A, X := E) \equiv A[E/X]$, where $A[E/X]$ denotes the result of simultaneously replacing occurrences of X in A with E . For a conditional with guard H and loop-free branches C_1 and C_2 ,

$$\begin{aligned} \mathbf{wlp}(A, \text{if } H \text{ then } C_1 \text{ else } C_2) \\ \equiv (H \rightarrow \mathbf{wlp}(A, C_1)) \wedge (\neg H \rightarrow \mathbf{wlp}(A, C_2)). \end{aligned}$$

Intuitively, the $\mathbf{wlp}$ rule for **if-then-else** asserts that the precondition of the selected branch must hold. Skip is straightforward, with $\mathbf{wlp}(A, \text{skip}) \equiv A$, while sequencing iterates $\mathbf{wlp}$ computation backward from the last command in the sequence.

Thus, when B is loop-free, all three verification conditions for Equation 4 can be written as FOL entailments. For the remainder of this section, we consider commands in *single-loop form*; that is, of the form in Equation 4, where B is loop-free.

In practice, it is convenient to search for *conjunctive invariants* I , which are invariants given as the conjunction of some set of smaller assertions, called *clauses*⁴. When an invariant is written as a conjunction of clauses, the verification conditions $(\mathbf{init})_I$ and $(\mathbf{maint})_I$ can be decomposed into *per-clause* checks:

$$\begin{array}{ll} PRE \models c, \text{ and} & (\mathbf{init})_c \\ I \wedge G \models \mathbf{wlp}(c, B). & (\mathbf{step})_{I,c} \end{array}$$

We refer to these as the *initialization* and *preservation* conditions, respectively. Observe that in the definition of $(\mathbf{step})_{I,c}$, the entire invariant I is available on the left-hand side of the entailment. A

³that is, containing no occurrences of **while**

⁴Clauses are often assumed to have a particular shape, such as being disjunctions of literals, but the WHIEL system uses “clause” and “assertion” interchangeably.

clause c is *preserved* by I if $(\mathbf{step})_{I,c}$ holds. The following proposition justifies this decomposition.

PROPOSITION 4.3. *Let $I = \bigwedge_{c \in S} c$ denote the conjunction of a finite set S of clauses. Then*

- (1) $(\mathbf{init})_I$ holds if and only if $(\mathbf{init})_c$ holds for all $c \in S$; and
- (2) $(\mathbf{maint})_I$ holds if and only if $(\mathbf{step})_{I,c}$ holds for all $c \in S$.

The *Houdini algorithm* [24] uses this decomposition to compute the *maximal inductive subset*⁵ of a finite set S of clauses. That is, it computes the largest subset $S' \subseteq S$ such that $(\mathbf{init})_{I'}$ and $(\mathbf{maint})_{I'}$ hold, where $I' = \bigwedge_{c \in S'} c$. If the resulting invariant I' also satisfies $(\mathbf{term})_{I'}$, then the Hoare triple is valid. WHIEL employs a modified version of the Houdini algorithm (Section 4.4) which is specialized to handle the *prophecy variables* and *leveled invariants* described in Sections 4.2 and 4.3, respectively.

4.2 Prophecy Variables

Consider again the triple in Equation 2. For this running example, we write G and B to denote the guard $T \neq S$ and the body $T := S$; $S := E \cup E \circ T$ of $\mathbf{TC}_{\text{WHIEL}}$, respectively. One initial attempt at providing an invariant is to use $c_0 \equiv S = E \cup E \circ T$, which is easily seen to satisfy $(\mathbf{init})_{c_0}$ and $(\mathbf{maint})_{c_0}$. However, termination fails:

$$(S = E \cup E \circ T) \wedge S = T \not\models T \circ T \subseteq T$$

is an invalid entailment. This is because c_0 and $S = T$ imply that T is a fixed point of the map $X \mapsto E \cup (E \circ X)$, but do not guarantee that T is transitive. Furthermore, the assertion $T \circ T \subseteq T$ itself is too strong to be an invariant; while it is initially true (since $T = \emptyset$), and it clearly satisfies the termination condition, it fails to be preserved in general during executions of the loop.

Prophecy Variables and Terminal Consequences. We conjecture that no invariant expressible in FOL over the execution schema $\{E, S, T\}$ exists for the Hoare triple in Equation 2. This seems to be related to the fact that FOL cannot express transitive closure: the postcondition concerns only the terminating value of T , which is precisely the transitive closure of E . To mitigate this apparent lack of expressivity, we introduce the following definitions.

Definition 4.4 (Prophecy variables and schema). Let C be a WHIEL command over execution schema Γ . For each $P \in \text{IDB}(C)$, introduce a fresh relation symbol $P^\infty \notin \Gamma$ of the same arity as P , called a *prophecy variable*. The *prophecy schema* of C is

$$\Gamma^\infty = \Gamma \cup \{P^\infty \mid P \in \text{IDB}(C)\}.$$

The intended interpretation of P^∞ is that it denotes the *terminal value* of P , its value at the end of a terminating run of the program C . Note that EDBs are not assigned prophecy variants because, by definition, their values are unchanged during the execution of C .

Definition 4.5 (Prophecy variants and terminal consequences). Let C be a WHIEL command over execution schema Γ . For a relation symbol $P \in \text{IDB}(C)$, its *prophecy variant* is the corresponding prophecy variable P^∞ . For an assertion A over Γ^∞ , its *terminal consequence* A^∞ is obtained by simultaneously replacing each occurrence of every $P \in \text{IDB}(C)$ with P^∞ . EDB symbols and prophecy variables already present in A are unchanged.

⁵This guarantee presupposes that the verification condition checks are conclusive.

WHIEL supports invariants over the prophecy schema of its inputs. Under the intended interpretation of prophecy variables, A^∞ expresses what A , if inductive, “says” about the terminal state. However, this alone does not increase the expressive power of the system; any assertion over Γ^∞ satisfying the three verification conditions **(init)**_I, **(maint)**_I, and **(term)**_I can be transformed into an invariant over the original schema Γ . To strengthen our system, we must modify the verification conditions themselves. The first way to do this is to hard-code the intended meaning of the prophecy variables in the termination condition. Keeping **(init)**_I and **(maint)**_I unchanged, we can replace the **(term)**_I check with

$$I \wedge \neg G \wedge \bigwedge_{P \in \text{IDB}(C)} P = P^\infty \models \text{POST}.$$

Equivalently, we can check

$$I^\infty \wedge \neg G^\infty \models \text{POST}^\infty. \quad \textbf{(term)}_I^\infty$$

The program does not contain occurrences of the prophecy symbols, and so the entailments **(init)**_I and **(maint)**_I must hold for *arbitrary relations* assigned to the prophecy variables; their intended meaning is applied only in the termination check.

Definition 4.6 (Prophecy invariant). A *prophecy invariant* for a Hoare triple of the form in Equation 4, where C is a WHIEL program over execution schema Γ , is an assertion I over Γ^∞ such that **(init)**_I, **(maint)**_I, and **(term)**_I are valid.

PROPOSITION 4.7 (PROPHECY RULE). *If a prophecy invariant exists for a Hoare triple of the form in Equation 4, then the triple is valid.*

4.3 Leveled Invariants

Proposition 4.7 enables us to construct an invariant for Equation 2. Consider the assertion $c_1 \equiv T \circ T^\infty \subseteq T^\infty$, which, unlike $T \circ T \subseteq T$, is true at the start of the TC_{WHIEL} loop and holds on every iteration (under the intended interpretation of T^∞). Furthermore, we have that $c_1^\infty \equiv \text{POST}^\infty \equiv T^\infty \circ T^\infty \subseteq T^\infty$, and therefore any conjunctive invariant I with c_1 as a clause suffices to ensure that **(term)**_I holds. However, more is needed since **(maint)**_{c₁} fails due to the fact that, while c_1 holds at the start of each loop iteration *under the intended interpretation*, Proposition 4.7 requires that **(maint)**_I holds for *arbitrary* values assigned to T^∞ , which is not the case.

We know from our earlier discussion that c_0 is inductive, and it gives some information about the terminating values of S and T . In particular, its terminal consequence is $c_0^\infty \equiv S^\infty = E \cup E \circ T^\infty$, while that of the negation of the loop guard, $\neg G^\infty$, is $T^\infty = S^\infty$. Together, these entail that $T^\infty = E \cup E \circ T^\infty$, and so

$$E \circ T^\infty \subseteq T^\infty. \quad (5)$$

This last fact suffices to preserve c_1 ; more precisely, we establish the entailment

$$c_0 \wedge c_1 \wedge \neg G^\infty \wedge c_0^\infty \models \text{wlp}(c_1, B). \quad (6)$$

Here, c_0 and c_1 should be interpreted as assumptions about the current values of the IDBs, while $\neg G^\infty \wedge c_0^\infty$ is an assumption about the terminating values. Since B first assigns $T := S$ and then updates only S , we have that $\text{wlp}(c_1, B) \equiv S \circ T^\infty \subseteq T^\infty$. Using c_0 , c_1 , and the derived fact in Equation 5, we obtain through standard reasoning steps that $S \circ T^\infty \subseteq T^\infty$, which establishes the preservation entailment in Equation 6. In general, a preservation proof may also

assume G (that the guard is true for the current values); we did not need to use this assumption in the above.

Intuitively, the argument proceeds in two stages. The first stage establishes that c_0 is inductive. The second stage uses the assumption that c_0 is inductive, allowing us to assume its terminal consequence c_0^∞ . This combines with the assumption $\neg G^\infty$, asserting that the guard is false for the terminal values, to derive the useful fact in Equation 5. This useful fact allows us to derive $S \circ T^\infty \subseteq T^\infty$, establishing that c_1 holds on each iteration of the TC_{WHIEL} loop, *given the terminal consequences c_0^∞ and $\neg G^\infty$* . Finally, the terminal consequence c_1^∞ suffices to prove the termination condition of Proposition 4.7. This suggests the following prophecy invariant:

$$I_{\text{TC}} \equiv c_0 \wedge ((\neg G^\infty \wedge c_0^\infty) \rightarrow c_1). \quad (7)$$

Leveled verification conditions. Consider the per-clause checks of Proposition 4.3 for each top-level conjunct of I_{TC} . For c_0 , these are

$$\text{PRE} \models c_0, \quad \text{and} \quad \textbf{(init)}_{c_0}$$

$$I_{\text{TC}} \wedge G \models \text{wlp}(c_0, B). \quad \textbf{(step)}_{I_{\text{TC}}, c_0}$$

For $d_1 \equiv (\neg G^\infty \wedge c_0^\infty) \rightarrow c_1$, they are

$$\text{PRE} \models (\neg G^\infty \wedge c_0^\infty) \rightarrow c_1, \quad \text{and} \quad \textbf{(init)}_{d_1}$$

$$I_{\text{TC}} \wedge G \models \text{wlp}((\neg G^\infty \wedge c_0^\infty) \rightarrow c_1, B). \quad \textbf{(step)}_{I_{\text{TC}}, d_1}$$

Since $\neg G^\infty$ and c_0^∞ do not contain IDBs, they are unchanged by taking the weakest liberal precondition with respect to B :

$$\text{wlp}((\neg G^\infty \wedge c_0^\infty) \rightarrow c_1, B) \equiv (\neg G^\infty \wedge c_0^\infty) \rightarrow \text{wlp}(c_1, B).$$

Consequently, we can simplify the checks of the second conjunct of I_{TC} to the equivalent entailments

$$\text{PRE} \wedge \neg G^\infty \wedge c_0^\infty \models c_1, \quad \text{and}$$

$$c_0 \wedge c_1 \wedge G \wedge \neg G^\infty \wedge c_0^\infty \models \text{wlp}(c_1, B).$$

Our previous argument establishes that **(init)**_{c₀} and **(step)**_{I_{TC}, c₀}, as well as the above equivalent versions of **(init)**_{d₁} and **(step)**_{I_{TC}, d₁}, are valid. Thus by Proposition 4.3, I_{TC} is inductive. Furthermore,

$$I_{\text{TC}}^\infty \equiv c_0^\infty \wedge ((\neg G^\infty \wedge c_0^\infty) \rightarrow c_1^\infty),$$

and we know that $c_1^\infty \equiv \text{POST}^\infty$, so

$$I_{\text{TC}}^\infty \wedge \neg G^\infty \models \text{POST}^\infty \quad \textbf{(term)}_{I_{\text{TC}}}^\infty$$

holds. Hence we can conclude by Proposition 4.7 that the Hoare triple in Equation 2 is valid.

Generalizing the argument. The form of the prophecy invariant I_{TC} can be generalized to support similar stage-based reasoning for *arbitrary* Hoare triples of the form in Equation 4, where the command C has execution schema Γ .

Definition 4.8. A *leveled candidate* is a tuple $\mathcal{A} = (S_0, \dots, S_m)$ of finite sets of clauses (called *levels*) over the prophecy schema Γ^∞ , where S_0 contains no occurrences of prophecy variables. We write $\bigcup \mathcal{A} = \bigcup_{j=0}^m S_j$ to denote the set of all clauses in $\mathcal{A}$. The *ladder assertion* of $\mathcal{A}$ with respect to C is the assertion

$$L_{\mathcal{A}} \equiv A_0 \wedge \bigwedge_{k=1}^m \left(\left(\neg G^\infty \wedge \bigwedge_{i < k} A_i^\infty \right) \rightarrow A_k \right), \quad (8)$$

where $A_k = \bigwedge_{c \in S_k} c$ and empty conjunctions denote true.

The invariant I_{TC} in Equation 7 is exactly the ladder assertion of the leveled candidate $\mathcal{A}_{TC} = (\{c_0\}, \{c_1\})$ with respect to $\mathbf{TC}_{\mathbf{WHIEL}}$. In the general case, we can also introduce modified verification conditions which match the argument for $\mathbf{TC}_{\mathbf{WHIEL}}$. For the level-0 clauses, these are the typical conditions:

$$\begin{aligned} PRE \models c, \quad \text{and} \quad & (\mathbf{init})_{\mathcal{A},c}^0 \\ G \wedge A_0 \models \mathbf{wlp}(c, B). \quad & (\mathbf{step})_{\mathcal{A},c}^0 \end{aligned}$$

For $j > 0$, the checks mirror the argument for $\mathbf{TC}_{\mathbf{WHIEL}}$. When checking initialization at a level S_j for $j > 0$, we may assume that the loop guard is false for the terminating values, and that all strictly lower levels satisfied their leveled initialization and preservation checks (and hence we may assume their terminal consequences). Thus the *level- k initialization* condition is

$$PRE \wedge \neg G^\infty \wedge \bigwedge_{i < k} A_i^\infty \models c. \quad (\mathbf{init})_{\mathcal{A},c}^k$$

The preservation check may assume that the guard is true at the current values, that it is false for the terminal values, that clauses at the current level and below are satisfied on the current iterations, and that all strictly lower levels satisfied their leveled initialization and preservation checks. Thus the *level- k preservation* condition is

$$G \wedge \bigwedge_{i \leq k} A_i \wedge \neg G^\infty \wedge \bigwedge_{i < k} A_i^\infty \models \mathbf{wlp}(c, B). \quad (\mathbf{step})_{\mathcal{A},c}^k$$

The termination condition $(\mathbf{term})_{\mathcal{A}}$ for a leveled candidate $\mathcal{A} = (S_0, \dots, S_m)$ is precisely $(\mathbf{term})_{\mathcal{A}}^\infty$, where $A = \bigwedge_{k \leq m} A_k$.

Definition 4.9. A leveled candidate $\mathcal{A} = (S_0, \dots, S_m)$ is an (*inductive*) *core* if $(\mathbf{init})_{\mathcal{A},c}^k$ and $(\mathbf{step})_{\mathcal{A},c}^k$ are valid for all $k \leq m$ and $c \in S_k$. It is a *leveled invariant* if $(\mathbf{term})_{\mathcal{A}}$ is valid as well.

PROPOSITION 4.10 (LADDER RULE). *If a leveled invariant $\mathcal{A}$ exists for a Hoare triple of the form in Equation 4, then the triple is valid. Furthermore, $L_{\mathcal{A}}$ is a prophecy invariant for the same Hoare triple.*

4.4 Leveled Houdini

The **LEVELEDHOUDINI** algorithm takes as input a single-loop Hoare triple τ of the form in Equation 4, a finite set of *candidate* clauses S , and a leveled candidate $\mathcal{K} = (K_0, \dots, K_m)$. All verification conditions and inductive cores in this section are relative to τ . Assuming $\mathcal{K}$ is an inductive core, **LEVELEDHOUDINI** constructs an inductive core $\mathcal{K}' = (K'_0, \dots, K'_n)$ with $m \leq n$ and $K_j \subseteq K'_j$ for all $j \leq m$. The output core may contain additional levels. We treat levels beyond the greatest tuple index of a leveled candidate as empty.

Intuitively, **LEVELEDHOUDINI** iteratively applies ordinary **HOUDINI** at each level j , starting from level 0, and applying the level- j initialization and preservation checks until reaching a fixed point. Once **LEVELEDHOUDINI** reaches a fixed point at a given level, that level is no longer changed during the run. For level-0, the checks are the usual per-clause checks. At a positive level, these checks come with the additional assumptions discussed previously, which are not provided by the standard **HOUDINI** algorithm. Clauses in the candidate set S which do not contain prophecy variables are first tried at level 0, while those containing prophecy variables are first tried at level 1. A clause which fails either level- j initialization or preservation is tried again at level $j + 1$, unless $j > 0$ and the updated core contains no clauses at level j or above. This is because

the additional assumptions may cause a clause to pass its checks at a higher level despite failing at a lower one.

Algorithm 1 **LEVELEDHOUDINI**($\tau, S, \mathcal{K}$)

Input: Single-loop Hoare triple τ ; finite candidate clause set S ;
Inductive core $\mathcal{K} = (K_0, \dots, K_m)$;

Output: An inductive core extending $\mathcal{K}$

```

1:  $P \leftarrow S \setminus \bigcup \mathcal{K}$ ;  $k \leftarrow 0$ 
2: while  $P \neq \emptyset$  do
3:    $R \leftarrow \{c \in P \mid \ell(c) \leq k\}$ ;  $R_{old} \leftarrow \emptyset$ 
4:   for  $c \in R$  do
5:     if  $(\mathbf{init})_{\mathcal{K},c}^k$  is invalid then  $R \leftarrow R \setminus \{c\}$ 
6:   while  $R \neq R_{old}$  do
7:      $R_{old} \leftarrow R$ 
8:      $\mathcal{A} \leftarrow \text{EXTEND}(\mathcal{K}, k, R)$ 
9:     for  $c \in R$  do
10:      if  $(\mathbf{step})_{\mathcal{A},c}^k$  is invalid then
11:         $R \leftarrow R \setminus \{c\}$ 
12:     break
13:    $\mathcal{K} \leftarrow \text{EXTEND}(\mathcal{K}, k, R)$ ;  $P \leftarrow P \setminus R$ 
14:   if  $k > 0$  and  $K_j = \emptyset$  for every  $j \geq k$  then break
15:    $k \leftarrow k + 1$ 
16: return  $\mathcal{K}$ 
```

Pseudocode for **LEVELEDHOUDINI** is provided in Algorithm 1. The outer loop iterates through levels in increasing order, with k denoting the current level. We write $\ell(c)$ to denote the initial level of clause c (either 0 or 1), as described above. The set P contains all clauses from S which do not already occur at some level in $\mathcal{K}$. On the k^{th} iteration of the outer loop, R contains those clauses to be considered for addition to level k of the core. The function $\text{EXTEND}(\mathcal{K}, k, R)$ returns the result of adding every clause in R to the k^{th} level of $\mathcal{K}$. The algorithm assumes access to an oracle for **WHIEL** assertion entailments. As in classical Houdini, the body of the outer loop first removes all clauses failing initialization for the corresponding level, followed by an inner loop removing all clauses which fail preservation. Clauses which survive this pruning process are added to the core at the corresponding level.

Correctness. Ordinary Houdini's pruning argument entails that each level also produces the largest subset of the clauses considered at that level which pass the initialization and preservation checks for that level. Furthermore, their inclusion at each level only adds to the hypotheses of the initialization and preservation checks for clauses already in the core. The inner loop always terminates because each iteration either removes a clause from R , or else the guard fails on the next iteration. The outer loop halts because any iteration for a level above m either removes a clause from P , or the loop is broken on line 14. Furthermore, if $(\mathbf{term})_{\mathcal{K}'}$ holds for the returned core $\mathcal{K}'$, then the input Hoare triple is valid by Proposition 4.10.

5 IMPLEMENTATION OVERVIEW

The **WHIEL** system implements the pipeline described in Figure 1. It takes as input a Hoare triple, conducts a search for a leveled invariant or a counterexample input state, and, when the search succeeds, certifies the solution in **LEAN**. **LEAN** certification serves as

a guarantee of soundness; consequently, Theorem 2.1 implies that WHIEL is incomplete. To mitigate the risk of non-termination, the implementation incorporates a number of resource limits. These include configurable time limits bounding the overall search time, the time allotted to VAMPIRE solver calls, the execution of candidate counterexamples, and the construction of LEAN certificates. Exhausting the overall search time leaves the triple unresolved, while exhausting the certification time leaves the verifier’s answer uncertified. A Rust-based controller manages the interactions between the LEAN library, VAMPIRE, and a Python harness for the LLM-based proposer. We describe the components below, with the experimental parameters deferred until Section 6.

5.1 LEAN Foundation and Preprocessing

WHIEL’s LEAN library formalizes database instances, syntax and semantics for RA, WHIEL assertions and commands, and soundness of the Hoare logic principles introduced in Section 4. It also implements several operations used by the synthesis algorithm, including **wlp** computation, preprocessing, and various syntactic operations on formulas such as prophecy variable substitution.

The preprocessor uses classical methods [30], which may introduce additional nullary relation symbols to the schema, to transform an arbitrary Hoare triple into *single-loop form*, that is, the form of Equation 4 with B loop-free. A LEAN theorem proves that the resulting triple is *equivalid*⁶ with the original triple. Consequently, the search algorithm seeks an invariant over the preprocessed triple, while the certifier transfers the result back to the original.

5.2 Neurosymbolic Search with AGENTHOUDINI

WHIEL’s search procedure is an interactive exchange between a *proposer* and a *verifier*. The verifier initially pushes the execution schema, the input Hoare triple, the prophecy schema, and the preprocessed Hoare triple to the proposer. On each round, the proposer has read-only access to tools provided by the verifier, including an assertion syntax well-formedness checker, a tool for inspecting a runtime record of verification condition check results and countermodels (discussed in more detail in Section 5.3), and a tool to evaluate assertions on concrete instances. For our implemented AGENTHOUDINI proposer, these tools are optional, and we evaluated the effect in Section 6. The proposer then makes a *Submission*, which is either a finite set of clauses over the prophecy schema (a *candidate clause set*), or a concrete database instance (a *candidate counterexample*). Note that we use *countermodel* to refer to an instance which proves invalidity of a WHIEL entailment, while we use *counterexample* to refer to an instance which proves invalidity of a Hoare triple.

An inductive core, initially empty, is maintained by the verifier and grows monotonically: once a clause is added to one of its levels, it is never removed during the remainder of the search. When a candidate clause set is submitted, the verifier runs an implementation of the LEVELEDHOUDINI algorithm to expand the inductive core (see Section 4.4). If this core passes the leveled candidate termination check, then the search ends and we proceed to certification. Otherwise, the updated inductive core and the set of clauses from the previous proposal not accepted to the core are sent back to the

proposer on the next round. A proposed clause c not admitted to the core by LEVELEDHOUDINI but to be reconsidered later is *pending*; such clauses will be reconsidered at level $\ell(c)$ on the next round in which a candidate set is submitted. The proposer has the option to request that pending clauses be dropped (i.e., not reconsidered).

Algorithm 2 WHIEL’s Search Algorithm

Input: Original triple τ ; preprocessed triple τ' ; search time budget
Output: Accepted core, validated counterexample, or unresolved

```

1: PUSH TASK( $\tau, \tau'$ )
2:  $\mathcal{K} \leftarrow (\emptyset)$ ; Pending  $\leftarrow \emptyset$ 
3: while search time budget remains do
4:   Submission  $\leftarrow$  PROPOSE( $\mathcal{K}$ , Pending)
5:   if Submission is a candidate clause set  $S$  then
6:      $\mathcal{K} \leftarrow$  LEVELEDHOUDINI( $\tau', S \cup$  Pending,  $\mathcal{K}$ )
7:     Pending  $\leftarrow (S \cup$  Pending)  $\setminus \mathcal{K}$ 
8:     If (term) $_{\mathcal{K}}$  = valid then return  $\mathcal{K}$ 
9:   else if Submission is a candidate counterexample  $s$  then
10:    If VALIDATE( $\tau, s$ ) succeeds then return  $s$ 
11: return unresolved

```

When a candidate counterexample is submitted, the verifier first checks that the instance satisfies the (original) precondition. If it does, then the verifier uses a LEAN-implemented WHIEL command interpreter to run the (original) command on the concrete state; if this halts, it checks if the output falsifies the (original) postcondition.

Pseudocode for our search procedure is provided in Algorithm 2, which runs after preprocessing and before certification⁷. The special case of this abstract search procedure in which proposals are given by an LLM agent is the AGENTHOUDINI algorithm.

5.3 Checking First-Order Validity with VAMPIRE

WHIEL uses VAMPIRE in the place of the finite validity oracle assumed by LEVELEDHOUDINI. This imposes some unavoidable limitations on the system. First, VAMPIRE checks validity of first-order entailments over *arbitrary* structures, whereas WHIEL entailments correspond to first-order entailments over *finite* structures. A valid FOL entailment is also finitely valid, but the reverse does not hold. Thus VAMPIRE is *sound* but *incomplete* for WHIEL entailments.

For each verification condition checked, LEAN code translates the WHIEL entailment into an equivalent⁸ first-order entailment over finite structures. This FOL entailment is then passed to VAMPIRE by the Rust controller to execute a concurrent proof and finite countermodel search. Both the proof search and finite countermodel search are subject to the same *solver time budget*. There are three possible outcomes: success (validity via proof), failure (invalidity via finite countermodel), or inconclusive. The latter outcome occurs when the solver time budget is exhausted or when VAMPIRE encounters an error, such as running out of memory. When pruning clauses in LEVELEDHOUDINI, our implementation treats inconclusive checks as failures. As a result, this also limits the theoretical guarantee that the core contains the largest inductive subset of clauses considered at each level. Similarly, inconclusive termination checks may also occur even when the core is actually sufficient for validity.

⁷The pseudocode omits solver retry policies and other optimization heuristics.

⁸Modulo details involving empty structures, which are handled in the implementation.

⁶Two Hoare triples are equivalid if either both are valid or both are invalid.

5.4 Solution Certification and Trusted Codebase

Certification occurs *after* the search procedure has returned either a leveled invariant or a validated counterexample. For the former, the certifier constructs LEAN files containing formalizations of the verification conditions. It then invokes VAMPIRE’s LEANCHECK, which produces a proof that is reconstructed in LEAN using the VAMLEAN tool⁹. These certified proof reconstructions are combined with theorems from the WHIEL LEAN library’s Hoare logic formalization and preprocessing correctness theorem to produce a validity proof for the input Hoare triple. Similarly, counterexamples accepted by the verifier have their validation re-run in the LEAN kernel, producing a proof of invalidity for the input Hoare triple. The emitted theorems in these LEAN *certificates* are named `input_hoare_triple_valid` or `input_hoare_triple_invalid`. Certification does not rely on an LLM agent, and solutions can be certified independently of the search process that produced them. The certifier rejects any dependencies of the emitted theorems on axioms other than LEAN’s standard axioms. A failure of proof reconstruction or counterexample replay during certification produces no certificate.

The purpose of the certifier is to produce a minimal *trusted code base*. A successfully certified solution for the WHIEL Hoare validity problem does not require trust in the search algorithm, Rust controller, proposer, VAMPIRE, VAMLEAN, or even the code that generates the certificate. Instead, the only requirement for confidence in a certified solution is trust in the LEAN kernel and trust in the fidelity of the LEAN library’s formalization of WHIEL commands and the definition of Hoare validity. To this end, we provide a file `AUDIT.md` in the WHIEL repository with a map for auditing the LEAN formalization.

6 EXPERIMENTS

We evaluate WHIEL on a benchmark of 86 verification tasks that we curated (Section 6.1) and ask the following questions: (1) **RQ1 (Comparison of WHIEL with Baselines)**: Does our method WHIEL solve tasks that the same LLM cannot solve by writing LEAN proofs directly? (2) **RQ2 (Effect of Skills and Tools)**: Do curated skills and MCP tools help the proposer solve more tasks, or solve them in fewer rounds, in less time, or at a lower cost? (3) **RQ3 (Impact of Stronger Models in the Proposer)**: When stronger models are used as a proposer, does WHIEL solve more tasks, and which tasks need them? (4) **RQ4 (Effect of Prophecy Variables)**: How tasks requiring prophecy variables compare with those that do not need them? (5) **RQ5 (Certification by WHIEL)**: What is the additional overhead for using the optional certifier provided by WHIEL?

6.1 Benchmark

To our knowledge, no existing benchmark targets the verification of imperative relational programs against relational specifications. Therefore, we construct a benchmark of 86 verification tasks. Those tasks are drawn from classical database problems (e.g., DATALOG programs [1], query reformulation using views [20], DRed [27]

⁹Our current system applies some small proof transformations to the generated proofs in order to improve compilation time and replace invocations of the `bv_decide` SAT checker with kernel-checked LRAT replay to avoid dependence on axioms introduced by native evaluation.

Table 1: The benchmark summary statistics.

(a) Tasks by postcondition type.

Type	Tasks	Valid	Invalid
Equivalence	64	52	12
Containment	18	14	4
Correctness	2	1	1
Implication	2	2	0
Total	86	69	17

(b) Structure of the tasks.

	Min	Median	Max
# While loops	1	2	6
# Relation symbols	3	9	32
# Assignments	4	12	43
# Invariant clauses*	2	10	33

*Over the reference invariants of the 69 valid tasks.

and the chase [5, 43]), real systems (e.g., PostgreSQL [57], NetworkX [28] and Soufflé [34]), and LLM-generated programs. Each task is a Hoare triple $\{P\} C \{Q\}$ over a WHIEL program, together with its ground-truth verdict: valid or invalid. The suite contains 69 valid and 17 invalid tasks and instantiates the problem classes of Section 3, including recursive query equivalence and containment, evaluation strategies, incremental view maintenance, and relational correctness properties. A detailed description of the benchmark and 86 tasks is available in the full version [59] and the benchmark is available in the WHIEL git repository [58].

Table 1a groups the tasks by the type of their postcondition. Most state query equivalence (64 tasks), e.g., the naive and semi-naive evaluation of a DATALOG program, query rewriting with views, or query containment (18 tasks). The remaining four state a property of a single computed relation, such as transitivity (correctness, 2 tasks), or combine claims through an implication or a disjunction (2 tasks). Table 1b summarizes the size of the tasks; 41 of the 69 reference invariants use prophecy relations, 34 of them in equivalence tasks.

Every task is backed by a LEAN certificate: for a valid task, the verification conditions of a reference invariant are proved in LEAN; for an invalid task, the LEAN kernel replays a counterexample. In our experiments, the proposer never sees the reference answers; we use them only as ground truth, to confirm the verdicts that WHIEL returns and to characterize the difficulty of the tasks.

6.2 Experimental Setup

Experiments ran on a Linux server with an AMD EPYC 7R13 processor (48 cores, 2.6 GHz) and 256 GB RAM. WHIEL uses LEAN 4 and VAMPIRE¹⁰. The proposer is an OpenAI Codex CLI agent [49] that calls remote model APIs from a sandbox hiding benchmark solutions and earlier results. Four tasks run in parallel, each with at most 12 single-threaded solver processes (8 VAMPIRE and 4 LEAN). **Methods.** We evaluate WHIEL, whose search is with AGENTHOUDINI (Section 5.2): an LLM agent proposes and the verifier checks. In the default configuration, which we call simply WHIEL throughout this section, the agent can only submit invariant clauses or a counterexample and receive the verifier’s feedback. We also built two extensions of this interface with *MCP tools* and *skills*, designed to help the agent exploit the verifier’s feedback and domain knowledge (details in the full version [59]). Each extension can be enabled independently, which gives four configurations: **WHIEL**, **WHIEL+Skills**, **WHIEL+Tools** and **WHIEL+Tools+Skills**. (1) **Tools** are six read-only tools (Section 5.2) that the verifier exposes to the agent through the *Model Context Protocol (MCP)* [2]. (2) **Skills** are six curated tutorials on constructing and repairing invariants, which the agent

¹⁰The WHIEL git repository [58] specifies the exact builds of both tools.

Table 2: Comparison of WHIEL with baselines (using GPT-5.5). Time, rounds, and cost are averaged over all 86 tasks.

Method	# Solved (%)			Per-task averages		
	Valid (69)	Invalid (17)	Total (86)	Time (s)	Rounds	Cost (\$)
WHIEL	57 (83%)	16 (94%)	73 (85%)	189	2.95	0.49
WHIEL-ONEROUND	32 (46%)	14 (82%)	46 (53%)	58	1	0.14
DIRECTLEAN	4 (6%)	16 (94%)	20 (23%)	398	–	2.08

reads on demand. Each is a written procedure: when it applies, what to inspect in the task, how to construct candidate clauses, and how to read the next failure. For example, Ladder strategy explains how to build a leveled invariant bottom-up, from facts that hold at every iteration to clauses that need facts about the exit state. In the configurations that provide them the prompt lists the available tools and skills with one-line descriptions, and the agent decides whether to use them.

Baselines. To our knowledge, no existing tool automatically verifies imperative relational programs with while loops, so we compare WHIEL against two baselines that use the same LLM (i.e., GPT-5.5). (i) *DIRECTLEAN*: an LLM agent receives the task as a LEAN Hoare-validity statement with the semantics, proof lemmas and proof interface, and may read the library sources, compile LEAN code and revise its proof in an isolated sandbox, without access to AGENTHOUDINI, the WHIEL verifier or the reference answers. It submits a verdict with a complete LEAN proof of the statement or of its negation, which counts only if the LEAN kernel checks it in full¹¹. (ii) *WHIEL-ONEROUND*: only the first round of each WHIEL run, one proposal followed by invariant or counterexample checking, without verifier feedback.

LLMs and budgets. Unless stated otherwise, the agent uses GPT-5.5 [50], to show the effectiveness of WHIEL with an LLM at moderate cost. In the model-scaling experiment (Section 6.5), the tasks that GPT-5.5 leaves unsolved are passed to stronger models, GPT-5.6 Sol [51] and then GPT-6 Astra [52]; each model attempts only the tasks left unsolved by the previous one, from scratch, so the coverage we report is cumulative. We set the reasoning effort to medium for all models. Each attempt has a 600 s search budget for the proposer’s turns and all verifier checks and ends when the verifier accepts an answer (invariant or counterexample) or when the budget runs out; DIRECTLEAN runs without a time limit.

Metrics. We report four metrics. (i) *Solved tasks*: the number and fraction of the 86 tasks that a method solves. WHIEL solves a task when its verifier accepts the answer (an invariant for validity or a counterexample for invalidity); DIRECTLEAN solves a task when LEAN accepts its proof. (ii) *Search time*: the elapsed time per task, which covers the agent’s turns and all checks but not certification. (iii) *Rounds*: the number of rounds per task, where a round is one submission from the proposer and its verification.

6.3 RQ1: Comparison of WHIEL with Baselines

WHIEL outperforms both baselines in effectiveness. As Table 2 shows, DIRECTLEAN solves 20 tasks (23%) even without a time limit. WHIEL solves 73 tasks (85%) in less than half the time (189 s

vs. 398 s) and at about a quarter of the cost (\$0.49 vs. \$2.08), and each of its verdicts rests on verification conditions proved by VAMPIRE or on a counterexample validated by the LEAN-implemented interpreter. DIRECTLEAN names the correct verdict for all 86 tasks. What it fails to produce is evidence for valid claims: it solves only 4 of the 69 valid tasks. Of its 66 unsolved tasks, 45 do not compile, 19 compile but fail the axiom audit, and 2 submit no proof.¹²

WHIEL-ONEROUND shows that even with one round, our framework does better than DIRECTLEAN. The model submits a single proposal, taking 15% of the time of DIRECTLEAN (58 s vs. 398 s) and 7% of its cost (\$0.14 vs. \$2.08), and it only has to propose candidate clauses or a concrete counterexample: LEVELEDHOUDINI keeps the inductive clauses, VAMPIRE discharges the verification conditions, and the LEAN-implemented interpreter validates counterexamples. This division of labor raises the solve rate from 23% to 53%. Iterating with verifier feedback adds 27 solved tasks, taking 3.1 rounds on average. In WHIEL with iterations and feedback, although the time increases, the success rate significantly increases even when skills and tools are not used.

6.4 RQ2: Effect of Skills and Tools

To evaluate the effect of the two extensions of WHIEL with tools and/or skills, we compare four configurations using GPT-5.5. As Figure 3 shows, WHIEL+Skills solves the most tasks over the full run, 77 of the 86 (90%), while WHIEL, WHIEL+Tools and WHIEL+Tools+Skills have similar performance within the same search time, with 73 (85%), 74 (86%) and 73 (85%) solved tasks. Of all tasks, 69 (80%) are solved by all four configurations and 7 (8%) by none of them. Figure 4 shows that the four configurations behave alike on the tasks they solve: 1.5–1.8 rounds, 105–137 s and 44–89 clause checks per task. Despite similar overall coverage, the configurations differ in how quickly they find answers. WHIEL+Skills solves 54 tasks in the first round, compared with 46–48 for the other configurations, and 66 by the second round, compared with 60 for WHIEL+Tools and 53 for WHIEL.

As Figure 4 shows, compared with WHIEL, all extensions reduce the avg. number of rounds and avg. clause checks. Compared to WHIEL+Tools, WHIEL+Skills is more effective as it reduces both avg. search time and avg. cost over WHIEL, while WHIEL+Tools increases them. In WHIEL+Skills, the model picks the skills that fit the task: it reads *delta coupling* in 75% of its model requests on tasks with semi-naïve evaluation and never on other tasks, and three of the five tasks that it solves and WHIEL does not are among them.

WHIEL+Tools+Skills does not perform better than WHIEL+Skills and increases avg. cost from \$0.46 to \$0.80. It lowers the solved tasks from 77 to 73 and increases the avg. rounds from 2.09 to 2.35. We see two likely reasons. First, these extensions overlap: both steer the model toward the missing clauses, skills by describing invariant patterns up front and tools by exposing concrete countermodels, so the second adds little guidance to the first. Second, their overheads add up. With both, the model makes 7.7 tool and skill calls per model request instead of 3.5 and processes 175k tokens per request instead of 79k; its largest context per request averages 34.5k tokens

¹¹That is, with no unproved placeholders (sorry) or extra axioms.

¹²Of the 19, five leave sorry placeholders, six declare the goal as a new axiom, five prove only the tautology $goal \vee \neg goal$, and three omit the answer theorem. sorry is a LEAN placeholder that closes any proof goal without proving it.

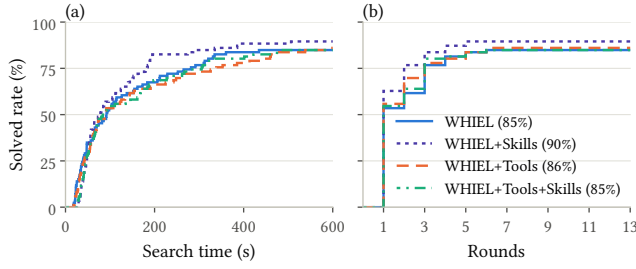


Figure 3: Solved rate of the four configurations of WHIEL, without and with (+ tools and/or skills) on 86 tasks; GPT-5.5.

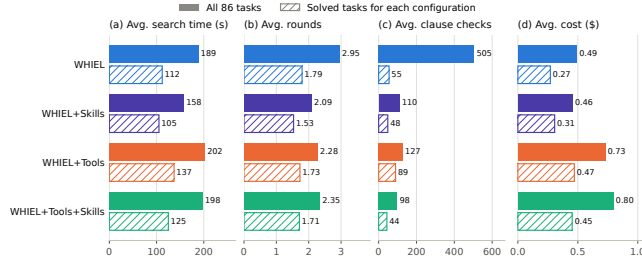


Figure 4: Average search time, rounds, clause checks and cost per task for the four WHIEL configurations over all 86 tasks (solid) and solved tasks (hatched); GPT-5.5. Avg. search time in (a) for all tasks includes 600 s for timed-out tasks.

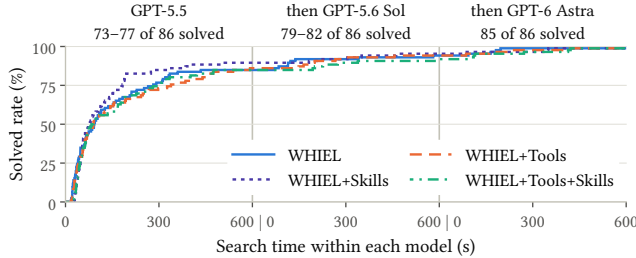


Figure 5: Solved rate and search time of the four WHIEL configurations; the tasks left unsolved by one model are passed to the next: GPT-5.5, then GPT-5.6 Sol, then GPT-6 Astra.

instead of 21.2k as tool replies accumulate. Each round then takes 84 s instead of 76 s. The longer context may also dilute useful hints.

6.5 RQ3: Impact of Stronger Models in Proposer

As shown in Figure 5, we run every task that GPT-5.5 model leaves unsolved, first with GPT-5.6 Sol, then with GPT-6 Astra. The coverage, i.e., the number of solves tasks, increases from 73 to 81 (GPT-5.6 Sol), and to 85 of the 86 tasks for WHIEL, and the other three configurations also reach 85 with Astra. The tasks that need a stronger model are the same at the hard end in every configuration: four tasks resist GPT-5.5 and Sol everywhere, and Astra solves three of them, namely red/blue alternating paths (linear vs. nonlinear), DRed deletion vs. recomputation, and Soufflé’s topological ordering (naive vs. semi-naive). The last one is invalid: GPT-5.5 and Sol run

out of time, whereas Astra submits a counterexample in its first model request in all four configurations.

6.6 RQ4: Effect of Prophecy Variables

One obvious source of difficulty in the tasks is **whether a prophecy variable is necessary** (Section 4.2); the success rates for different models are shown below:

	GPT 5.5	GPT 5.6	GPT 6	Unsolved
Prophecy (total 41)	30 (73%)	7 (17%)	3 (7%)	1 (3%)
No prophecy (total 28)	27 (96%)	1 (4%)	-	-

As expected, if prophecy variables are not needed, the success rate is high even for GPT 5.5 and the overall success rate is 100%. For some tasks requiring prophecy variables, a clause over terminal values cannot be read off either program: the proposer has to guess which facts about the terminal state hold. The hardest tasks nest such facts. Both tasks whose every known invariant has clauses at level 2 (Section 4.3) need a stronger model. e.g., GPT 5.6 Sol solves *Red/blue closure vs. DATALOG path views*, and no model solves *Warshall sweep vs. PostgreSQL recursive union* (program details in [59]) within the budget, although GPT 6 Astra proposes its nested facts in its first round. There, VAMPIRE seems to be the bottleneck: even given the reference invariant, the verifier needs 333 s of the 600 s budget to prove it. We defer analysis based on other properties of the programs and tasks to the full version.

6.7 RQ5: Certification by WHIEL

Running the certifier is an optional process in WHIEL, depending on the need of the application. After all search experiments were complete, we ran the certifier on the 340 verifier-accepted answers from all four configurations utilized during search (85 per configuration). We utilized a 300 s overall time limit, a 300 s internal VAMPIRE call allowance, and memory limits of 10 GB per process. Each certification attempt used 12 physical cores and 12 solver workers, with two attempts running concurrently on disjoint cores. This certified 293/340 answers (86.2%): 225/272 validity answers and all 68 invalidity answers. Among successful attempts, the certification time had mean 72.2 s, median 53.1 s, and maximum 295.8 s; sampled max single-process memory usage had mean 2.753 GB, median 2.540 GB, and maximum 7.591 GB. The remaining attempts resulted in 41 timeouts and six memory-limit failures. These resource failures of the certifier reflect known difficulties for LEAN with elaborating and type-checking large VAMPLEAN-generated proofs [9]; improving resource usage here is a target for future work.

7 CONCLUSION AND FUTURE WORK

We have presented the WHIEL system, which combines LLM-guided invariant and counterexample synthesis with a symbolic verification pipeline employing a novel LEVELEDHOUDINI algorithm and an independent certification procedure. Our experiments demonstrate the value of this neurosymbolic architecture on a curated benchmark of examples motivated by hard database problems. Future work includes developing a more comprehensive relational programming benchmark, refining the agent-based synthesis pipeline, and reducing the time and memory requirements of the certifier. We also plan to study extensions of the language with fresh-value creation, arithmetic, aggregation, and semiring-annotated data.

REFERENCES

- [1] Serge Abiteboul, Richard Hull, and Victor Vianu. 1995. *Foundations of Databases*. Addison-Wesley.
- [2] Anthropic. 2024. Introducing the Model Context Protocol. <https://www.anthropic.com/news/model-context-protocol>. Specification at <https://modelcontextprotocol.io>.
- [3] Molham Aref, Paolo Guagliardo, George Kastrinis, Leonid Libkin, Victor Marsault, Wim Martens, Mary McGrath, Filip Murlak, Nathaniel Nystrom, Liat Peterfreund, Allison Rogers, Cristina Sirangelo, Domagoj Vrgoč, David Zhao, and Abdul Zreika. 2025. Rel: A Programming Language for Relational Data. In *Companion of the 2025 International Conference on Management of Data* (Berlin, Germany) (SIGMOD/PODS '25). Association for Computing Machinery, New York, NY, USA, 283–296. <https://doi.org/10.1145/3722212.3724450>
- [4] Luigi Bellomarini, Emanuel Sallinger, and Georg Gottlob. 2018. The Vadalogue system: datalog-based reasoning for knowledge graphs. *Proc. VLDB Endow.* 11, 9 (May 2018), 975–987. <https://doi.org/10.14778/3213880.3213888>
- [5] Michael Benedikt, George Konstantinidis, Giansalvatore Mecca, Boris Motik, Paolo Papotti, Donatello Santoro, and Efthymia Tsamoura. 2017. Benchmarking the Chase. In *Proceedings of the 36th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems (PODS '17)*. Association for Computing Machinery, New York, NY, USA, 37–52. <https://doi.org/10.1145/3034786.3034796>
- [6] Rudolf Berghammer, Peter Höfner, and Insa Stucke. 2014. Automated Verification of Relational While-Programs. In *Relational and Algebraic Methods in Computer Science - 14th International Conference, RAMICS 2014, Marienstatt, Germany, April 28-May 1, 2014. Proceedings (Lecture Notes in Computer Science)*, Peter Höfner, Peter Jipsen, Wolfram Kahl, and Martin Eric Müller (Eds.), Vol. 8428. Springer, 173–190. https://doi.org/10.1007/978-3-319-06251-8_11
- [7] Maciej Besta, Marc Fischer, Vasiliki Kalavri, Michael Kapralov, and Torsten Hoefer. 2023. Practice of Streaming Processing of Dynamic Graphs: Concepts, Models, and Systems. *IEEE Trans. Parallel Distributed Syst.* 34, 6 (2023), 1860–1876. <https://doi.org/10.1109/TPDS.2021.3131677>
- [8] Paul Biberstein, Ziyang Li, Joseph Devietti, and Mayur Naik. 2025. Lobster: A GPU-Accelerated Framework for Neurosymbolic Programming. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1* (Pittsburgh, PA, USA) (ASPLOS '26). Association for Computing Machinery, New York, NY, USA, 147–162. <https://doi.org/10.1145/3760250.3762232>
- [9] Jonas Bodingbauer, Márton Hajdu, Laura Kovács, Axel Polaczek, and Michael Rawson. 2026. Lean on Vampire Proofs. In *17th International Conference on Interactive Theorem Proving (ITP 2026) (Leibniz International Proceedings in Informatics (LIPIcs))*, Ekaterina Komendantskaya and Tobias Nipkow (Eds.), Vol. 382. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 36:1–36:9. <https://doi.org/10.4230/LIPIcs.ITP.2026.36>
- [10] Angela Bonifati and Stefania Dumbra. 2019. Graph Queries: From Theory to Practice. *SIGMOD Rec.* 47, 4 (May 2019), 5–16. <https://doi.org/10.1145/3335409.3335411>
- [11] Marco A. Casanova and Philip A. Bernstein. 1980. A Formal System for Reasoning about Programs Accessing a Relational Database. *ACM Trans. Program. Lang. Syst.* 2, 3 (1980), 386–414. <https://doi.org/10.1145/357103.357111>
- [12] Stefano Ceri, Georg Gottlob, and Letizia Tanca. 1989. What you Always Wanted to Know About Datalog (And Never Dared to Ask). *IEEE Trans. Knowl. Data Eng.* 1, 1 (1989), 146–166. <https://doi.org/10.1109/69.43410>
- [13] Shumo Chu, Chenglong Wang, Konstantin Weitz, and Alvin Cheung. 2017. Cosette: An Automated Prover for SQL. In *8th Biennial Conference on Innovative Data Systems Research, CIDR 2017, Chaminade, CA, USA, January 8-11, 2017, Online Proceedings*. www.cidrdb.org. <http://cidrdb.org/cidr2017/papers/p51-chu-cidr17.pdf>
- [14] E. F. Codd. 1970. A relational model of data for large shared data banks. *Commun. ACM* 13, 6 (June 1970), 377–387. <https://doi.org/10.1145/362384.362685>
- [15] Stephen A. Cook. 1974. An Observation on Time-Storage Trade Off. *J. Comput. Syst. Sci.* 9, 3 (1974), 308–316. [https://doi.org/10.1016/S0022-0000\(74\)80046-2](https://doi.org/10.1016/S0022-0000(74)80046-2)
- [16] Leonardo de Moura and Sebastian Ullrich. 2021. The Lean 4 Theorem Prover and Programming Language. In *Automated Deduction - CADE 28 - 28th International Conference on Automated Deduction, Virtual Event, July 12-15, 2021, Proceedings (Lecture Notes in Computer Science)*, André Platzer and Geoff Sutcliffe (Eds.), Vol. 12699. Springer, 625–635. https://doi.org/10.1007/978-3-030-79876-5_37
- [17] Mert Demirel, Leon Musolf, and Liyuan Yang. 2026. Writing Code vs. Shipping Code: Productivity Effects Across Generations of AI Coding Tools. Working Paper 35275. National Bureau of Economic Research. <https://doi.org/10.3386/w35275>
- [18] Edsger W. Dijkstra. 1976. *A Discipline of Programming*. Prentice-Hall. <https://www.worldcat.org/oclc/01958445>
- [19] Haoran Ding, Zhaoguo Wang, Yicun Yang, Dexin Zhang, Zhenglin Xu, Haibo Chen, Ruzica Piskac, and Jinyang Li. 2023. Proving Query Equivalence Using Linear Integer Arithmetic. *Proc. ACM Manag. Data* 1, 4, Article 227 (Dec. 2023), 26 pages. <https://doi.org/10.1145/3626768>
- [20] Oliver M. Duschka and Michael R. Genesereth. 1997. Answering recursive queries using views. In *Proceedings of the Sixteenth ACM SIGACT-SIGMOD-SIGART Symposium on Principles of Database Systems* (Tucson, Arizona, USA) (PODS '97). Association for Computing Machinery, New York, NY, USA, 109–116. <https://doi.org/10.1145/263661.263674>
- [21] Wenfei Fan. 2022. Big Graphs: Challenges and Opportunities. *Proc. VLDB Endow.* 15, 12 (2022), 3782–3797. <https://doi.org/10.14778/3554821.3554899>
- [22] Wenfei Fan, Chao Tian, Ruiqi Xu, Qiang Yin, Wenyuan Yu, and Jingren Zhou. 2021. Incrementalizing Graph Algorithms. In *SIGMOD '21: International Conference on Management of Data, Virtual Event, China, June 20-25, 2021*, Guoliang Li, Zhanhui Li, Stratos Idreos, and Divesh Srivastava (Eds.). ACM, 459–471. <https://doi.org/10.1145/3448016.3452796>
- [23] Raul Castro Fernandez, Aaron J. Elmore, Michael J. Franklin, Sanjay Krishnan, and Chenhao Tan. 2023. How Large Language Models Will Disrupt Data Management. *Proc. VLDB Endow.* 16, 11 (2023), 3302–3309. <https://doi.org/10.14778/3611479.3611527>
- [24] Cormac Flanagan and K. Rustan M. Leino. 2001. Houdini, an Annotation Assistant for ESC/Java. In *Proceedings of the International Symposium of Formal Methods Europe on Formal Methods for Increasing Software Productivity (FME '01)*. Springer-Verlag, Berlin, Heidelberg, 500–517.
- [25] Nadime Francis, Alastair Green, Paolo Guagliardo, Leonid Libkin, Tobias Linddaaker, Victor Marsault, Stefan Plantikow, Mats Rydberg, Petra Selmer, and Andrés Taylor. 2018. Cypher: An Evolving Query Language for Property Graphs. In *Proceedings of the 2018 International Conference on Management of Data, SIGMOD Conference 2018, Houston, TX, USA, June 10-15, 2018*, Gautam Das, Christopher M. Jermaine, and Philip A. Bernstein (Eds.). ACM, 1433–1445. <https://doi.org/10.1145/3183713.3190657>
- [26] Amélie Gheerbrant, Leonid Libkin, Liat Peterfreund, and Alexandra Rogova. 2025. GQL and SQL/PGQ: Theoretical Models and Expressive Power. *Proc. VLDB Endow.* 18, 6 (2025), 1798–1810. <https://doi.org/10.14778/3725688.3725707>
- [27] Ashish Gupta, Inderpal Singh Mumick, and V. S. Subrahmanian. 1993. Maintaining Views Incrementally. In *Proceedings of the 1993 ACM SIGMOD International Conference on Management of Data (SIGMOD '93)*. Association for Computing Machinery, New York, NY, USA, 157–166. <https://doi.org/10.1145/170035.170066>
- [28] Aric A. Hagberg, Daniel A. Schult, and Pieter J. Swart. 2008. Exploring Network Structure, Dynamics, and Function using NetworkX. In *Proceedings of the 7th Python in Science Conference (SciPy 2008)*, Gaël Varoquaux, Travis Vaught, and Jarrod Millman (Eds.). Pasadena, CA, USA, 11–15. <https://doi.org/10.25080/TCWV9851>
- [29] Soonbo Han and Zachary G. Ives. 2024. Implementation Strategies for Views over Property Graphs. *Proc. ACM Manag. Data* 2, 3 (2024), 146. <https://doi.org/10.1145/3654949>
- [30] David Harel. 1980. On Folk Theorems. *Commun. ACM* 23, 7 (1980), 379–389. <https://doi.org/10.1145/358886.358892>
- [31] Yang He, Ruijie Fang, Isl Dillig, and Yuepeng Wang. 2025. Graphiti: Bridging Graph and Relational Database Queries. *Proc. ACM Program. Lang.* 9, PLDI, Article 216 (June 2025), 25 pages. <https://doi.org/10.1145/3729319>
- [32] Yang He, Pinhan Zhao, Xinyu Wang, and Yuepeng Wang. 2024. VeriEQL: Bounded Equivalence Verification for Complex SQL Queries with Integrity Constraints. *Proc. ACM Program. Lang.* 8, OOPSLA1, Article 132 (April 2024), 29 pages. <https://doi.org/10.1145/3649849>
- [33] Shachar Itzhaky, Tomer Kotek, Noam Rinetzky, Mooly Sagiv, Orr Tamir, Helmut Veith, and Florian Zuleger. 2017. On the Automated Verification of Web Applications with Embedded SQL. In *20th International Conference on Database Theory, ICDT 2017, Venice, Italy, March 21-24, 2017 (LIPIcs)*, Michael Benedikt and Giorgio Orsi (Eds.), Vol. 68. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 16:1–16:18. <https://doi.org/10.4230/LIPIcs.ICDT.2017.16>
- [34] Herbert Jordan, Bernhard Scholz, and Pavle Subotić. 2016. Soufflé: On Synthesis of Program Analyzers. In *Computer Aided Verification*, Swarat Chaudhuri and Azadeh Farzan (Eds.). Springer International Publishing, Cham, 422–430.
- [35] Ralf Jung, Rodolphe Lepigre, Gaurav Parthasarathy, Marianna Rapoport, Amin Timany, Derek Dreyer, and Bart Jacobs. 2020. The future is ours: prophecy variables in separation logic. *Proc. ACM Program. Lang.* 4, POPL (2020), 45:1–45:32. <https://doi.org/10.1145/3371113>
- [36] Adharsh Kamath, Nausheen Mohammed, Aditya Senthilnathan, Saikat Chakraborty, Pantazis Deligiannis, Shuvendu K. Lahiri, Akash Lal, Aseem Rastogi, Subhajit Roy, and Rahul Sharma. 2024. Leveraging LLMs for Program Verification. In *2024 Formal Methods in Computer-Aided Design (FMCAD)*. 107–118. https://doi.org/10.34727/2024/isbn.978-3-85448-065-5_16
- [37] Jingyu Ke, Jinyang Li, and Guoqiang Li. 2026. Logos: Certified Order-Sensitive SQL Rewrites with Mechanized Semantics and LLM Guidance. arXiv:2608.15709 [cs.DB]. <https://arxiv.org/abs/2608.15709>
- [38] Laura Kovács and Andrei Voronkov. 2013. First-Order Theorem Proving and Vampire. In *Computer Aided Verification - 25th International Conference, CAV 2013, Saint Petersburg, Russia, July 13-19, 2013. Proceedings (Lecture Notes in Computer Science)*, Natasha Sharygina and Helmut Veith (Eds.), Vol. 8044. Springer, 1–35. https://doi.org/10.1007/978-3-642-39799-8_1

- [39] Leslie Lamport and Stephan Merz. 2022. Prophecy Made Simple. *ACM Trans. Program. Lang. Syst.* 44, 2 (2022), 6:1–6:27. <https://doi.org/10.1145/3492545>
- [40] Kukjin Lee, Anshuman Dutt, Vivek Narasayya, and Surajit Chaudhuri. 2026. Verified LLM-Based Query Rewriting for Microsoft SQL Server. *Proc. VLDB Endow.* 19, 12 (Sept. 2026), 4594–4597. <https://doi.org/10.14778/3827998.3828074>
- [41] Ziyang Li, Jiani Huang, and Mayur Naik. 2023. Scallop: A Language for Neurosymbolic Programming. *Proc. ACM Program. Lang.* 7, PLDI, Article 166 (June 2023), 25 pages. <https://doi.org/10.1145/3591280>
- [42] Boon Thau Loo, Tyson Condie, Minos Garofalakis, David E. Gay, Joseph M. Hellerstein, Petros Maniatis, Raghu Ramakrishnan, Timothy Roscoe, and Ion Stoica. 2009. Declarative networking. *Commun. ACM* 52, 11 (Nov. 2009), 87–95. <https://doi.org/10.1145/1592761.1592785>
- [43] David Maier, Alberto O. Mendelzon, and Yehoshua Sagiv. 1979. Testing Implications of Data Dependencies. *ACM Transactions on Database Systems* 4, 4 (1979), 455–469. <https://doi.org/10.1145/320107.320115>
- [44] Makai Mann, Ahmed Irfan, Alberto Griggio, Oded Padon, and Clark W. Barrett. 2022. Counterexample-Guided Prophecy for Model Checking Modulo the Theory of Arrays. *Log. Methods Comput. Sci.* 18, 3 (2022). [https://doi.org/10.46298/LMCS-18\(3:26\)2022](https://doi.org/10.46298/LMCS-18(3:26)2022)
- [45] René Mazala. 2002. *Infinite Games*. Springer Berlin Heidelberg, Berlin, Heidelberg, 23–38. https://doi.org/10.1007/3-540-36387-4_2
- [46] William McCune. 2005–2010. Prover9 and Mace4. (2005–2010). <http://www.cs.unm.edu/~mccune/prover9/>.
- [47] Kenneth L. McMillan. 2018. Eager Abstraction for Symbolic Model Checking. In *Computer Aided Verification - 30th International Conference, CAV 2018, Held as Part of the Federated Logic Conference, FloC 2018, Oxford, UK, July 14–17, 2018, Proceedings, Part I (Lecture Notes in Computer Science)*, Hana Chockler and Georg Weissenbacher (Eds.), Vol. 10981. Springer, 191–208. https://doi.org/10.1007/978-3-319-96145-3_11
- [48] Frank McSherry, Derek Gordon Murray, Rebecca Isaacs, and Michael Isard. 2013. Differential Dataflow. In *Sixth Biennial Conference on Innovative Data Systems Research, CIDR 2013, Asilomar, CA, USA, January 6–9, 2013, Online Proceedings*. www.cidrdb.org, 12. http://cidrdb.org/cidr2013/Papers/CIDR13_Paper111.pdf
- [49] OpenAI. 2026. Codex CLI. <https://github.com/openai/codex>. Versions 0.148.0 and 0.154.0.
- [50] OpenAI. 2026. GPT-5.5: model documentation. <https://developers.openai.com/api/docs/models/gpt-5.5>.
- [51] OpenAI. 2026. GPT-5.6 Sol: model documentation. <https://developers.openai.com/api/docs/models/gpt-5.6-sol>.
- [52] OpenAI. 2026. GPT-6 Astra: model documentation. <https://developers.openai.com/api/docs/models/gpt-6-astra>.
- [53] OpenAI. 2026. On the Navier-Stokes Millennium Prize Problem. <https://openai.com/index/navier-stokes-solution/>. Accessed September 28, 2026.
- [54] Muhammad A. A. Pirzada, Giles Reger, Ahmed Bhayat, and Lucas C. Cordeiro. 2024. LLM-Generated Invariants for Bounded Model Checking Without Loop Unrolling. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (Sacramento, CA, USA) (ASE '24)*. Association for Computing Machinery, New York, NY, USA, 1395–1407. <https://doi.org/10.1145/3691620.3695512>
- [55] Lei Tang, Wensheng Dou, Yingying Zheng, Lijie Xu, Wei Wang, Jun Wei, and Tao Huang. 2025. Proving Cypher Query Equivalence. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. 3931–3944. <https://doi.org/10.1109/ICDE65448.2025.00293>
- [56] The Rocq Development Team. 2025. *The Rocq Prover*. <https://doi.org/10.5281/zenodo.17473943>
- [57] The PostgreSQL Global Development Group. 2026. PostgreSQL. <https://www.postgresql.org>.
- [58] <https://github.com/JesseComer/whiel>. 2026. Whiel.
- [59] <https://jessecomer.github.io/files/whiel.pdf>. 2026. Whiel: A Neurosymbolic Platform for Certified Relational Program Verification.
- [60] Jin Wang, Jiacheng Wu, Mingda Li, Jiaqi Gu, Ariyam Das, and Carlo Zaniolo. 2021. Formal semantics and high performance in declarative machine learning using Datalog. *VLDB J.* 30, 5 (2021), 859–881. <https://doi.org/10.1007/S00778-021-00665-6>
- [61] Shuxian Wang, Sicheng Pan, and Alvin Cheung. 2024. QED: A Powerful Query Equivalence Decider for SQL. *Proc. VLDB Endow.* 17, 11 (July 2024), 3602–3614. <https://doi.org/10.14778/3681954.3682024>
- [62] Yuepeng Wang, Isil Dillig, Shuwendu K. Lahiri, and William R. Cook. 2018. Verifying equivalence of database-driven applications. *Proc. ACM Program. Lang.* 2, POPL (2018), 56:1–56:29. <https://doi.org/10.1145/3158144>
- [63] John Whaley, Dzintars Avots, Michael Carbin, and Monica S. Lam. 2005. Using Datalog with Binary Decision Diagrams for Program Analysis. In *Programming Languages and Systems*, Kwangkeun Yi (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 97–118.
- [64] Peng Zhang, Aaron Gember-Jacobson, Yueshang Zuo, Yuhao Huang, Xu Liu, and Hao Li. 2022. Differential Network Analysis. In *19th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2022, Renton, WA, USA, April 4–6, 2022*, Amar Phanishayee and Vyas Sekar (Eds.). USENIX Association, 601–615. <https://www.usenix.org/conference/nsdi22/presentation/zhang-peng>
- [65] Peng Zhang, Yuhao Huang, Aaron Gember-Jacobson, Wenbo Shi, Xu Liu, Hongkun Yang, and Zhiqiang Zuo. 2020. Incremental Network Configuration Verification. In *HotNets '20: The 19th ACM Workshop on Hot Topics in Networks, Virtual Event, USA, November 4–6, 2020*, Ben Y. Zhao, Heather Zheng, Harsha V. Madhyastha, and Venkat N. Padmanabhan (Eds.). ACM, 81–87. <https://doi.org/10.1145/3422604.3425936>